Preface

Most Rust programmers learn lifetimes the same way: by fixing errors. The compiler says that something "does not live long enough", you add an `&`, a `.clone()` or a `'a`, and the error goes away. After a while the fixes become reflexes, but the reasons stay vague, and every so often an error appears that no reflex fixes: "implementation of `Fn` is not general enough", "returning this value requires that `'1` must outlive `'static`", "hidden type captures lifetime that does not appear in bounds". This book is about the reasons.

It starts from a small, precise model: a lifetime is a *region*, a set of points in a function body, and a signature is a *contract* that the body and every caller are checked against. Almost everything else follows from those two ideas plus a handful of defaults that the compiler applies when you leave a lifetime out. The book makes every one of those defaults visible: what elision writes for you, what `'_` means in each position, why `Box<dyn Trait>` means `Box<dyn Trait + 'static>`, what an `impl Trait` return type captures, and why `F: Fn(&str)` is secretly a higher-ranked bound. Then it applies the model to the places where lifetimes are hardest to see: generic associated types, async code, interior mutability, `unsafe` code and API design.

Who this book is for

You write Rust: ownership and borrowing, structs and enums, traits and generics, closures and iterators are familiar, and you have met lifetime errors that you could fix without being able to explain. You do not need to know anything about type theory or the internals of the compiler. When the book uses a concept such as variance, a higher-ranked bound or an early-bound lifetime, it explains it from the beginning, with programs.

How this book is verified

Every program in this book is a file in the accompanying `code/` directory, a Cargo workspace that uses only the standard library, and every listing is included from those files when the book is built. The compiler is pinned to **Rust 1.99.0** with **edition 2024**.

The book makes two kinds of claims about programs, and both are checked mechanically:

- "**This compiles.**" The program is a runnable example in `code/examples/`. It must build without a single warning and run to completion, and its output must match the output shown in the book, which was captured from a real run.
- "**This is rejected.**" The program is in `code/compile_fail/`. It must fail with the error codes or messages it declares, and the diagnostics shown in the book are the compiler's own output,

captured verbatim. Nothing in a diagnostic is retyped or edited, with two exceptions: the trailing "aborting due to ..." lines are removed, and paths into your local copy of the standard library are written in the machine-independent form `/rustc/<commit>/library/...` that rustc uses when that copy is not installed.

A script, `tools/run_examples.py`, recompiles every program and compares every output and diagnostic with the snapshots in the book. Appendix A explains how to run it, and how to use it to see what changes when you try a newer compiler.

This matters more for lifetimes than for most topics. The rules have changed over time (non-lexical lifetimes in 2018, new capture rules for `impl Trait` in edition 2024, new lints and elision fixes in recent releases), and diagnostics improve with nearly every release. Much of what you will find online describes an older compiler or an older edition. Everything in this book describes Rust 1.99 with edition 2024; when an older edition behaves differently and you are likely to meet code written for it, the text says so and shows both.

Some features are not stable yet: closure lifetime binders, Polonius (the next borrow checker), bounds on higher-ranked lifetimes. The book describes them where they matter, says that they are unstable, and gives the date of the nightly compiler that was checked. Their status will change; check it before you rely on it.

How to read the listings

Each listing is introduced in the text, and the text says whether it compiles. Listings come in three kinds:

Fence	Comes from	Means
`rust` from `code/examples/`	a runnable example	compiles without warnings; its output follows in a plain block
`rust` from `code/compile_fail/`	a program that must be rejected	does not compile; the diagnostics follow
`rust` marked as not compiled	written in the text	an expansion ("the compiler reads this as ..."), a signature from another crate, or unstable syntax

The first time a listing appears, the text names its file. Listings often show only part of a file (the part that matters); the line numbers in a diagnostic always refer to the whole file, so they may not match the excerpt above them. Open the file when you want the full context, and run it:

```
cd code
cargo run --example ch09_callee_picks
```

Diagnostics are part of the lesson. The book reads them with you, label by label, because rustc's lifetime diagnostics are precise once you know their vocabulary: `'1` and `'2` are names the compiler invents for anonymous lifetimes, "for any lifetime" marks a requirement and "for some specific lifetime" marks what you have, and the arrows point at the exact places where a borrow starts, is used and must end.

How the book is organized

Part I, The Model (Chapters 1–5), builds the foundations: regions and non-lexical lifetimes; whose borrow a method returns; the elision rules and why the compiler refuses to guess; subtyping and variance; and the everyday mechanics of reborrows, temporaries, drops and the limits of today's borrow checker.

Part II, Lifetimes in Signatures (Chapters 6–9), covers the lifetimes that appear in real APIs: the anonymous lifetime `'_`; the two meanings of `'static`; trait objects and `impl Trait`, whose lifetime defaults and captures changed in edition 2024; and higher-ranked trait bounds, `for<'a>`.

Part III, Going Further (Chapters 10–14), applies the model to generic associated types and lending iterators, async code, interior mutability and its guards, `unsafe` code, and finally to the design of APIs: when to borrow, when to own, and how many lifetime parameters a type needs.

The **appendices** are for reference: setting up and verifying the examples, a catalogue of every lifetime-related error in the book, a quick reference, a glossary and a list of further reading.

Three threads run through all of it. *The signature is the contract*: a body is checked against its signature, and callers are checked against the signature and nothing else. *Who picks the lifetime?*: a lifetime parameter is picked by the caller, a higher-ranked lifetime by the callee, at each use. And *what the compiler writes for you*: every default is shown expanded, so that you can see the lifetime that was there all along.

Every chapter opens with a list of what you will learn and closes with a summary and exercises. The exercises are meant to be done with the compiler at hand: predict, then check.

Acknowledgments

This book grew out of an earlier, much shorter field guide on lifetimes, the anonymous lifetime and `'static`, written for rustc 1.75 and edition 2021. Its material was reviewed against Rust 1.99, corrected where the language or the compiler had moved on, and extended into the chapters that follow.

The book relies on the work of the people who document Rust: *The Rust Programming Language*, *The Rust Reference*, *The Rustonomicon*, the *rustc development guide* and the Rust blog. Where it describes a rule, it checks the rule against the compiler, and where the documentation and the compiler disagree, it says so. Appendix E lists the sources.